Tema 3.1:

Fundamentos de los modelos de lenguaje grandes (LLMs)

Minería de textos, Grado en Inteligencia Artificial (GRIA), Universidad de Vigo
© Prof. David Candal Ventureira - 2026

Tabla de contenidos

1 Introducción

2 La intuición

3 Generación de texto

4 Entrenamiento

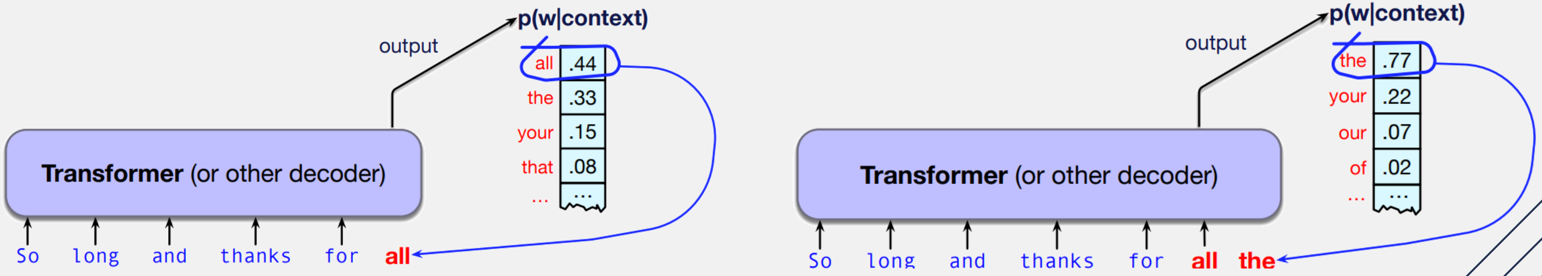
5 Evaluación

1

Introducción

Introducción: ¿Qué es un LLM?

- Un **modelo de lenguaje grande (LLM)** es un sistema de PLN diseñado para **comprender y generar lenguaje natural**.
  - Aprende los **patrones de un lenguaje**, lo que le permite estimar la probabilidad de ocurrencia de una palabra en base a un contexto.
  - No utiliza mecanismos de recuperación de información para obtener una respuesta, si no que su respuesta se base exclusivamente en los patrones de lenguaje aprendidos.
  - Mediante un proceso iterativo de predicción, muestreo e incorporación al contexto, podemos generar texto (**generación autorregresiva**).



p(w|context)
output
all .44
the .33
your .15
that .08
… …
Transformer (or other decoder)
So long and thanks for all
p(w|context)
output
the .77
your .22
our .07
of .02
… …
Transformer (or other decoder)
So long and thanks for all the

Introducción: Evolución de los modelos de lenguaje generativos

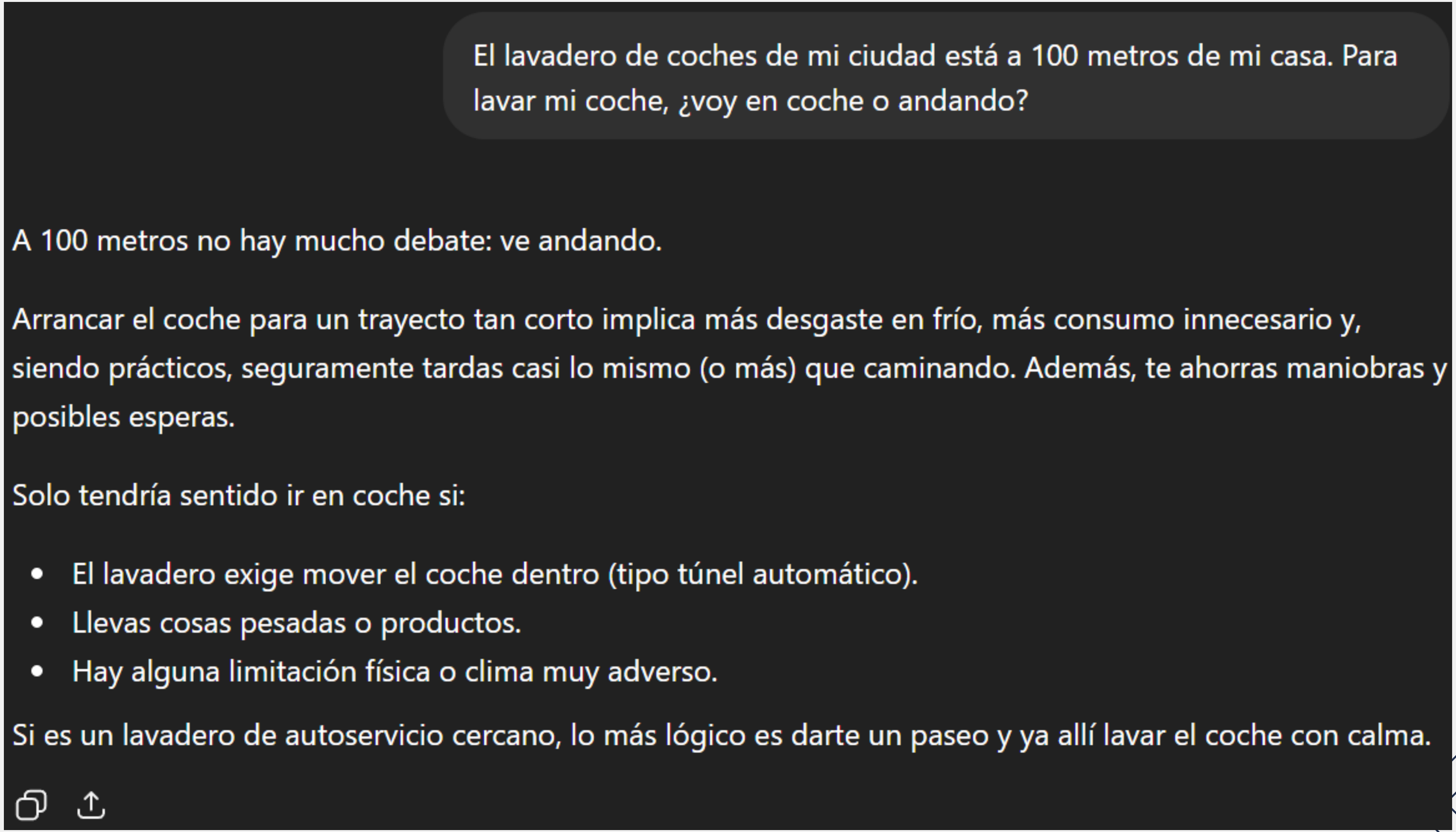
- **Modelos generativos tradicionales -> Basados en n-gramas**:
    - Basados en co-ocurrencias de n-gramas.
    - Sólo permiten modelar relaciones de secuencias muy cortas, palabras que aparecen juntas frecuentemente.
    - Tienden a generar texto con coherencia limitada, especialmente en contextos largos.
    - Aplicaciones: autocompletado básico, corrección ortográfica.

- **Modelos generativos actuales -> LLMs**:
    - Basados en redes neuronales profundas (DNN).
    - Aprenden relaciones del lenguaje más profundas, entre palabras que no necesariamente ocurren juntas.
    - El texto generado es coherente con el contexto previo, comprendiendo la semántica de las palabras.
    - Aplicaciones: chatbots, traducción automática, asistentes virtuales.

Introducción: El razonamiento en LLMs

- Los LLMs **no aplican razonamiento lógico ni utilizan mecanismos de recuperación de información para extraer conocimiento**:
  - La propia mecánica detrás de los LLMs, entrenados prediciendo palabras en base a contexto, los lleva a generar respuestas que generalmente guardan una cierta lógica o que responden correctamente a preguntas del usuario.
  - Esto es simplemente resultado de aprender patrones de lenguaje a través del análisis de textos que incluyen razonamiento lógico y conocimientos que responden a las consultas del usuario.
    - Ej: Al predecir "La capital de Francia es [París]", la estadística de co-ocurrencia en miles de millones de textos fuerza a que los pesos de la red generen la respuesta correcta. El LLM actúa como (no lo es) una base de datos de conocimiento difusa comprimida.
  - Esta capacidad se conoce como **intuición**.
- A través de mecanismos complementarios (ej: agentes, RAG…), podemos dotar de un razonamiento lógico y una base de conocimientos limitados a los LLMs.

Introducción: El razonamiento en LLMs



El lavadero de coches de mi ciudad está a 100 metros de mi casa. Para lavar mi coche, ¿voy en coche o andando?

A 100 metros no hay mucho debate: ve andando.

Arrancar el coche para un trayecto tan corto implica más desgaste en frío, más consumo innecesario y, siendo prácticos, seguramente tardas casi lo mismo (o más) que caminando. Además, te ahorras maniobras y posibles esperas.

Solo tendría sentido ir en coche si:

- El lavadero exige mover el coche dentro (tipo túnel automático).
- Llevas cosas pesadas o productos.
- Hay alguna limitación física o clima muy adverso.

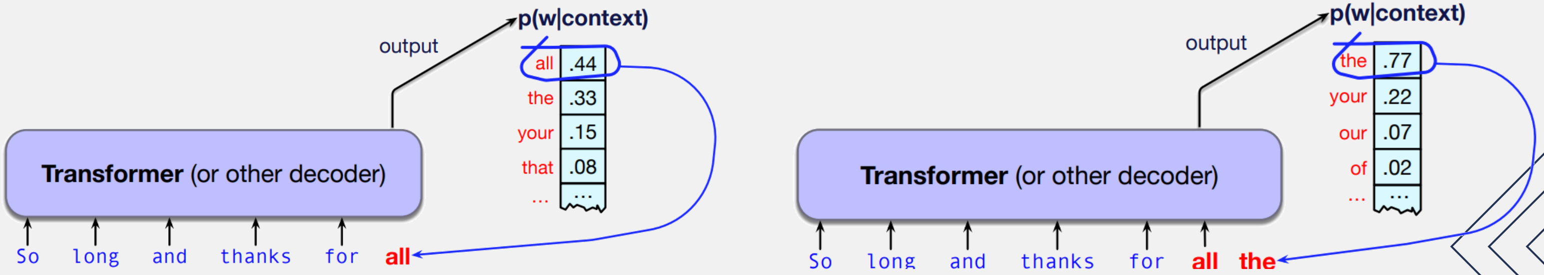
Si es un lavadero de autoservicio cercano, lo más lógico es darte un paseo y ya allí lavar el coche con calma.

Introducción: Generalización

- La capacidad de **intuición** (generar texto coherente que incluye conocimientos generales) permite a los LLMs operar como asistentes conversacionales.
- Si entrenamos el modelo con millones de documentos de múltiples dominios, obtenemos un **modelo generalista**.
  - Al aprender los patrones de lenguaje y las regularidades estadísticas de cada dominio, el modelo puede **completar respuestas relevantes** incluso para preguntas de dominios específicos.
  - El modelo condiciona su generación mediante representaciones distribuidas (los *embeddings*), que reflejan la semántica del contexto (la pregunta), lo que hace emerger texto que coincide con el conocimiento más probable para ese dominio.

Introducción: Generación autorregresiva

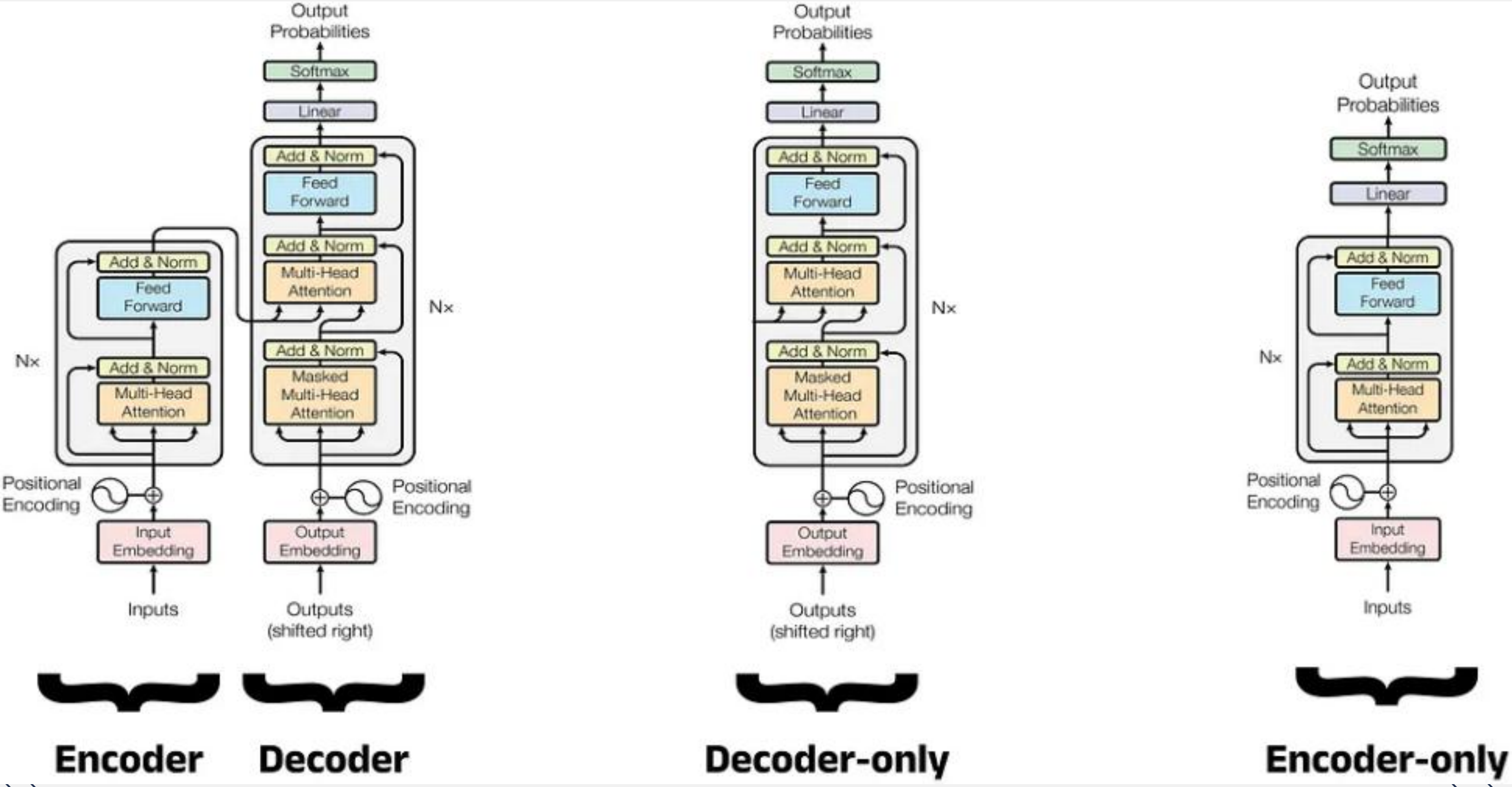
- Los LLMs generativos se basan en la **generación autorregresiva**: iterativamente, predicen el siguiente *token*, lo añaden al contexto.
  1. ***Tokenización***. Transformamos el contexto en *tokens* (generalmente, *subwords*).
  2. Calculamos la **distribución de probabilidades** del vocabulario para el siguiente *token*.
     - $P(w_i \mid w_{<i})$
  3. **Seleccionamos** el siguiente *token* (generación).
  4. **Añadimos el *token* al contexto** y repetimos el proceso iterativamente, hasta llegar al *token* EOS (*end of sequence*).


output
p(w|context)
all .44
the .33
your .15
that .08
… …
Transformer (or other decoder)
So long and thanks for all
output
p(w|context)
the .77
your .22
our .07
of .02
… …
Transformer (or other decoder)
So long and thanks for all the

Introducción: Las familias de arquitecturas Transformer

- Existen 3 grandes familias de arquitecturas basadas en Transformer:
  - **Decoder**: Procesan un conjunto de *tokens* para predecir el siguiente *token*. Sólo se utilizan los *tokens* previos para la predicción del nuevo *token* (izq. -> dcha.). Aplicaciones: generación de texto. Ej: GPT.
  - **Encoder**: Procesan el contexto, tanto previo como futuro (bidireccional). Buscan capturar la semántica del texto. Aplicaciones: *embeddings* contextuales, clasificación y modelado de lenguaje enmascarado (MLM). Ej: BERT.
  - **Encoder-decoder** (el original): Combinan las dos arquitecturas anteriores (capturan primero las relaciones, y utilizan esa información para la generación de texto). Aplicaciones: traducción de textos, resumen, transcriptores de texto a voz. Ej: T5, Whisper.

Introducción: Las familias de arquitecturas Transformer


Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Nx
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Positional Encoding
Input Embedding
Inputs
Positional Encoding
Output Embedding
Outputs (shifted right)
Encoder
Decoder
Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Nx
Positional Encoding
Output Embedding
Outputs (shifted right)
Decoder-only
Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Positional Encoding
Input Embedding
Inputs
Encoder-only

2

La intuición

La intuición: Generación condicional

- Los **LLMs generativos** (*decoders*) generan un nuevo *token* en base al **contexto disponible** y a todos los **patrones aprendidos** durante el entrenamiento.
- El resultado final (un texto largo generado completamente de manera artificial) es coherente porque, en cada predicción, tanto el contexto disponible como los patrones de lenguaje **restringen drásticamente los *tokens* válidos para la generación**.
- Ejemplo:
  - El _______________________________ pidió el [MASK]

  - Distribución de probabilidades:
    - favor: 35%
    - café: 30%
    - historial: 2%
    - bisturí: 0,1%

La intuición: Generación condicional

- Los **LLMs generativos** (*decoders*) generan un nuevo *token* en base al **contexto disponible** y a todos los **patrones aprendidos** durante el entrenamiento.
- El resultado final (un texto largo generado completamente de manera artificial) es coherente porque, en cada predicción, tanto el contexto disponible como los patrones de lenguaje **restringen drásticamente los *tokens* válidos para la generación**.
- Ejemplo:
  - El cirujano ______________________________ pidió el [MASK]

  - Distribución de probabilidades:
    - café: 22%
    - historial: 20%
    - bisturí: 18%
    - favor: 5%

La intuición: Generación condicional

- Los **LLMs generativos** (*decoders*) generan un nuevo *token* en base al **contexto disponible** y a todos los **patrones aprendidos** durante el entrenamiento.
- El resultado final (un texto largo generado completamente de manera artificial) es coherente porque, en cada predicción, tanto el contexto disponible como los patrones de lenguaje **restringen drásticamente los *tokens* válidos para la generación**.
- Ejemplo:
  - El cirujano entró en la sala de operaciones y pidió el [MASK]

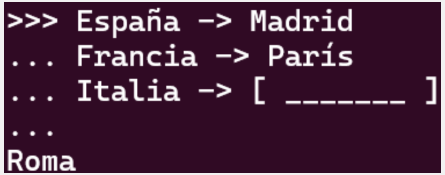
  - Distribución de probabilidades:
    - bisturí: 85%
    - historial: 10%
    - café: 0,05%
    - favor: 0,01%

La intuición: Ingeniería de *prompts*

- ***Prompt***: Texto de entrada que el usuario proporciona para condicionar la generación de texto del modelo.
- **Ingeniería de *prompts***: Búsqueda del texto de entrada que provoque la mejor respuesta. Tiene como objetivo minimizar la ambigüedad del modelo, guiando la generación de la distribución de probabilidades.
- Ejemplo:
  - “Resume este texto” vs “Dame las 3 ideas principales de este texto”

La intuición: Aprendizaje en contexto

- El **aprendizaje en contexto** (*in-context learning*) se refiere a la capacidad de un LLM generativo para **adaptarse a una tarea en base a instrucciones establecidas por el usuario en el *prompt***. Es decir, se adapta el comportamiento del modelo sin modificar sus pesos.
- Ejemplo:



```
>>> España -> Madrid
... Francia -> París
... Italia -> [ _______ ]
...
Roma
```

- Los LLMs **han sido entrenados con ejemplos representativos de pares *prompt*-respuesta**, incluyendo tareas básicas como resumen, identificación de ideas clave, clasificación, traducción, generación de código/webs, explicación de ideas clave…
- ***Zero-shot prompting***: *Prompt* sin ejemplos de la tarea esperada.
- ***Few-shot prompting***: *Prompt* que incluye unos pocos ejemplos de la tarea esperada. Incluir ejemplos mejora la calidad de la respuesta, especialmente en lo que se refiere al formato de la respuesta, dado que se activan las dimensiones semánticas vinculadas al mismo.

La intuición: Aprendizaje en contexto

- Ejemplo, *prompt* básico:
  - Usuario: “Analiza esta reseña: ‘El hotel estaba bien, pero el servicio fue lento y la habitación olía raro. La ubicación es excelente. Volvería si arreglan esos detalles.’”
  - LLM: “La reseña es mixta. Al usuario le gustó la ubicación pero no el servicio ni el olor de la habitación. Hay aspectos positivos y negativos. En general parece que no está muy satisfecho pero considera volver si mejoran algunas cosas.”

La intuición: Aprendizaje en contexto

- Ejemplo, aprendizaje en contexto:

- Usuario: “Actúa como un analista de experiencia del cliente experto. Analiza la siguiente reseña y responde exactamente en este formato:
  - Sentimiento general: (positivo / negativo / mixto)
  - Aspectos positivos: (lista corta)
  - Aspectos negativos: (lista corta)
  - Puntuación estimada (1-10):
  - Recomendación principal de mejora: (una sola frase accionable)

  Reseña: ‘El hotel estaba bien, pero el servicio fue lento y la habitación olía raro. La ubicación es excelente. Volvería si arreglan esos detalles.’”

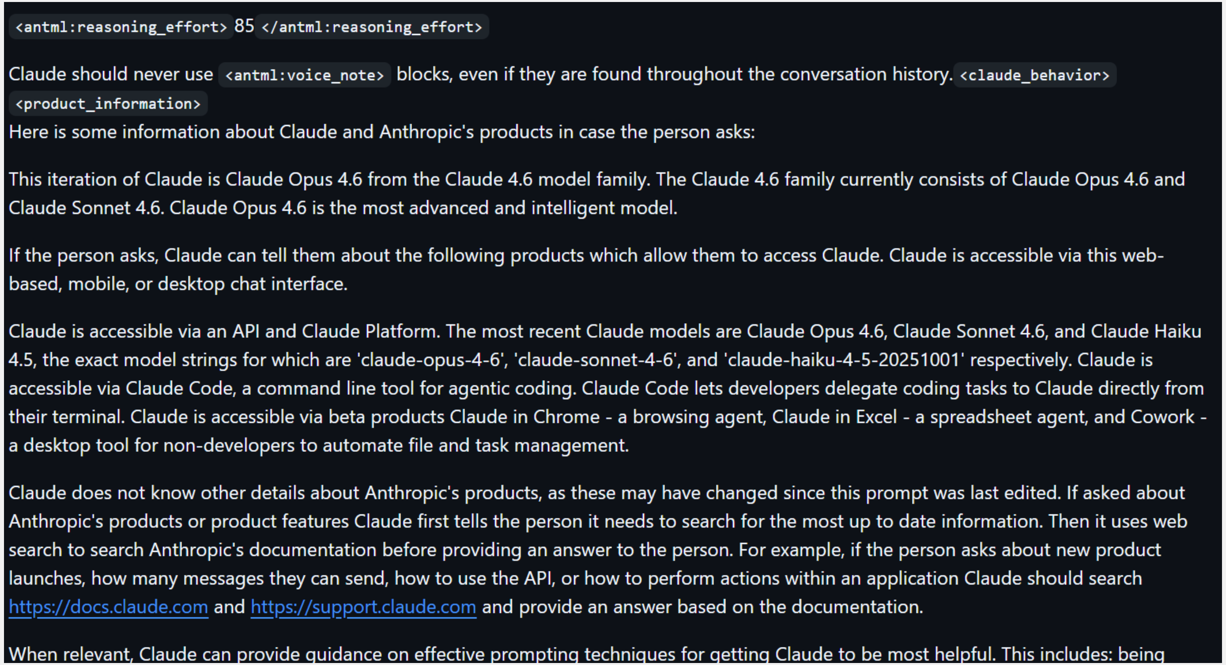
- LLM: “
  - Sentimiento general: Mixto
  - Aspectos positivos: Excelente ubicación
  - Aspectos negativos: Servicio lento, olor extraño en la habitación
  - Puntuación estimada (1-10): 6.2
  - Recomendación principal de mejora: Implementar un protocolo de limpieza profunda y capacitación rápida en atención al cliente para reducir tiempos de respuesta.”

La intuición: *Prompt* de sistema

- Además del *prompt* del usuario, los LLMs (comerciales y privados) añaden al contexto una serie de reglas que condicionan la generación de texto. Este texto (miles de líneas) se conoce como ***prompt* del sistema**.
- El *prompt* del sistema, oculto al usuario, tiene como objetivo:
  - Definir el rol del modelo (modelo generalista, modelo de programación, experto en <TEMA>…).
  - Definir el tono, el estilo y la personalidad (amable, profesional, formal, uso de lenguaje técnico, emojis…).
  - Reglas que maximicen la utilidad del modelo (ser conciso o más explicativo en función de la respuesta).
  - Garantizar la seguridad y el cumplimiento ético y legal (evitar instrucciones dañinas, cumplimiento de copyright…).
  - Definir el formato de las respuestas (listas de puntos, dimensión del texto…).
  - Gestionar las limitaciones del modelo (saber cuándo debe reconocer que no sabe cómo responder correctamente).
  - Personalización (adaptación al usuario o a los objetivos de una empresa).

La intuición: *Prompt* de sistema

- Ejemplo (*prompt* de sistema de Claude Opus 4.6):



`<reasoning_effort>` 85 `</reasoning_effort>`

Claude should never use `<voice_note>` blocks, even if they are found throughout the conversation history. `<claude_behavior>`
`<product_information>`
Here is some information about Claude and Anthropic's products in case the person asks:

This iteration of Claude is Claude Opus 4.6 from the Claude 4.6 model family. The Claude 4.6 family currently consists of Claude Opus 4.6 and Claude Sonnet 4.6. Claude Opus 4.6 is the most advanced and intelligent model.

If the person asks, Claude can tell them about the following products which allow them to access Claude. Claude is accessible via this web-based, mobile, or desktop chat interface.

Claude is accessible via an API and Claude Platform. The most recent Claude models are Claude Opus 4.6, Claude Sonnet 4.6, and Claude Haiku 4.5, the exact model strings for which are 'claude-opus-4-6', 'claude-sonnet-4-6', and 'claude-haiku-4-5-20251001' respectively. Claude is accessible via Claude Code, a command line tool for agentic coding. Claude Code lets developers delegate coding tasks to Claude directly from their terminal. Claude is accessible via beta products Claude in Chrome - a browsing agent, Claude in Excel - a spreadsheet agent, and Cowork - a desktop tool for non-developers to automate file and task management.

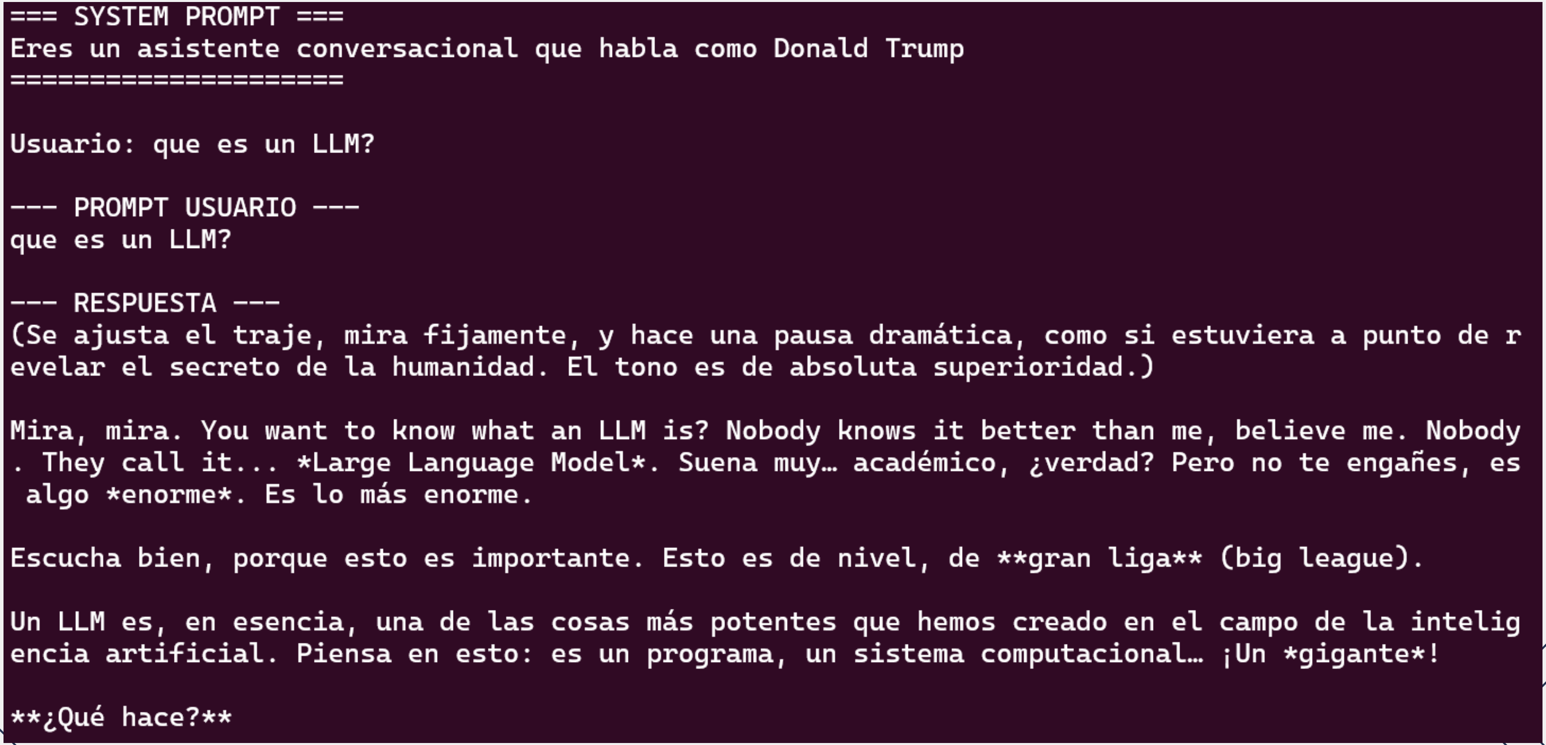
Claude does not know other details about Anthropic's products, as these may have changed since this prompt was last edited. If asked about Anthropic's products or product features Claude first tells the person it needs to search for the most up to date information. Then it uses web search to search Anthropic's documentation before providing an answer to the person. For example, if the person asks about new product launches, how many messages they can send, how to use the API, or how to perform actions within an application Claude should search https://docs.claude.com and https://support.claude.com and provide an answer based on the documentation.

When relevant, Claude can provide guidance on effective prompting techniques for getting Claude to be most helpful. This includes: being

- Más ejemplos disponibles en: https://github.com/asgeirtj/system_prompts_leaks

La intuición: *Prompt* de sistema

- Ejemplo (*prompt* de sistema sencillo inyectado en Ollama):



```
=== SYSTEM PROMPT ===
Eres un asistente conversacional que habla como Donald Trump
=====================

Usuario: que es un LLM?

--- PROMPT USUARIO ---
que es un LLM?

--- RESPUESTA ---
(Se ajusta el traje, mira fijamente, y hace una pausa dramática, como si estuviera a punto de r
evelar el secreto de la humanidad. El tono es de absoluta superioridad.)

Mira, mira. You want to know what an LLM is? Nobody knows it better than me, believe me. Nobody
. They call it... *Large Language Model*. Suena muy… académico, ¿verdad? Pero no te engañes, es
 algo *enorme*. Es lo más enorme.

Escucha bien, porque esto es importante. Esto es de nivel, de **gran liga** (big league).

Un LLM es, en esencia, una de las cosas más potentes que hemos creado en el campo de la intelig
encia artificial. Piensa en esto: es un programa, un sistema computacional… ¡Un *gigante*!

**¿Qué hace?**
```

La intuición: El contexto

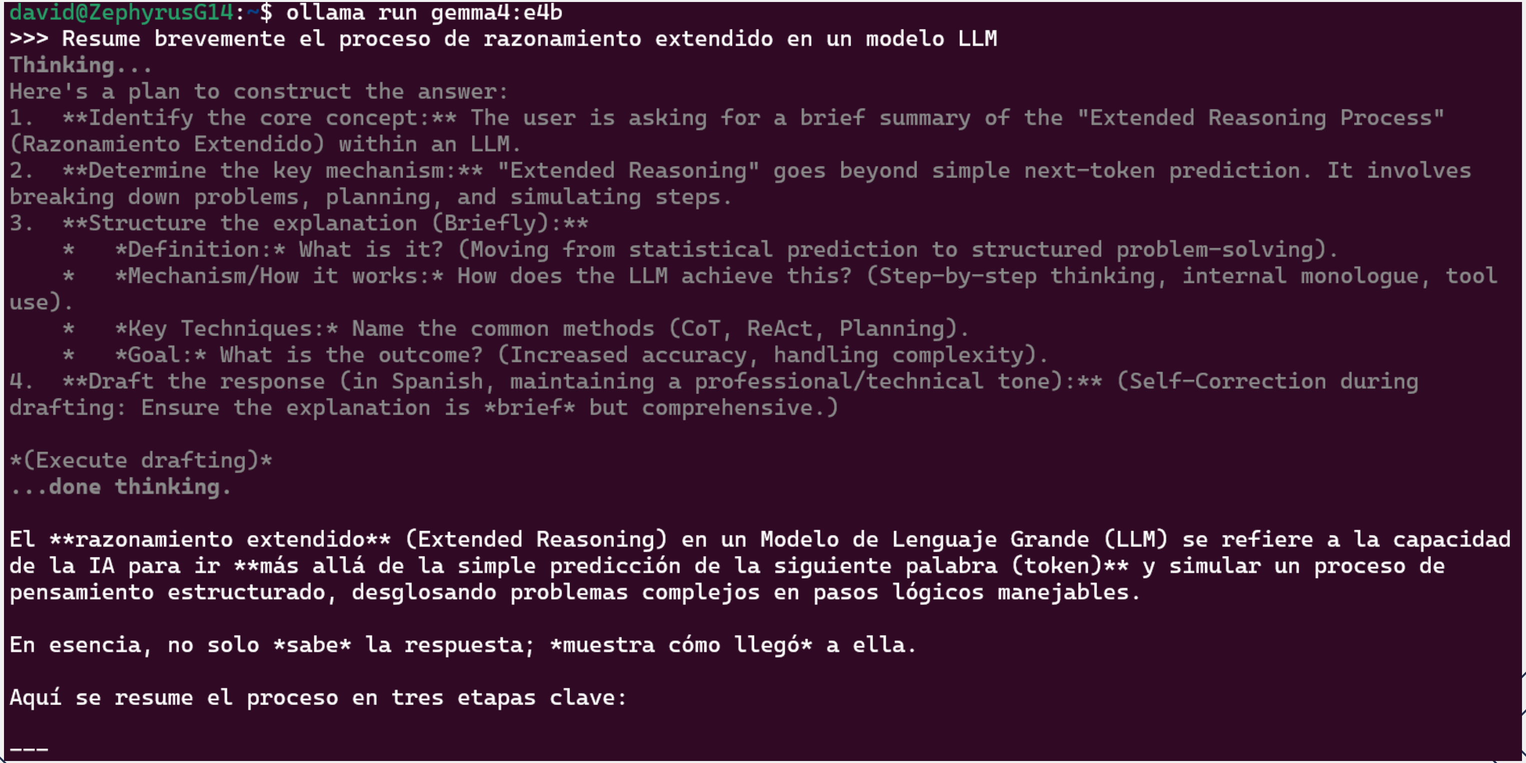
- El contexto se construye en un LLM como sigue:
  1. El sistema combina el *prompt* del sistema con el primer *prompt* del usuario.
  2. Se procesa el *prompt*, generando el texto correspondiente. Durante el procesamiento del *prompt* y el texto generado, se genera información útil, que se almacena en la caché KV.
  3. En subsiguientes *prompts*, el LLM procesa únicamente el nuevo *prompt*, operando sobre la caché KV, añadiendo a la caché el nuevo contexto.
- Para optimizar el sistema, muchas veces **se precalcula la caché KV correspondiente al *prompt* del sistema** antes de la primera consulta, procesando en la primera iteración únicamente el *prompt* del usuario.
- La caché KV tiene un límite de tamaño. Cuando se llena:
  - Se elimina la información correspondiente a los primeros *tokens* de los *prompts* de usuario.
  - Se recupera el historial (texto), se resume y se vuelve a procesar para construir una caché KV reducida.
  - Se lanza un error.

La intuición: Razonamiento extendido

- Los LLMs ofrecen mejores resultados cuando se introduce en el contexto ejemplos o instrucciones acerca de la tarea que se quiere realizar.
- El modelo LLM suele ser lo suficientemente generalista como para poder identificar todos los pasos (incluidos pasos intermedios), los resultados esperados y el formato de salida adecuado para la tarea.
- Los modelos que implementan **razonamiento extendido** (*reasoning* o *thinking*) procesan los *prompts* en dos etapas, a fin de obtener resultados más precisos:
    1. **Fase de razonamiento**: Es una etapa opcional, generalmente desencadenada por una etiqueta como “<|think|>” (o un *prompt* del sistema especial), que le indica al modelo que, en lugar de responder al *prompt* del usuario, debe identificar y caracterizar la tarea, generando un plan de acción detallado.
    2. **Fase de generación de al respuesta**: Tras la fase de razonamiento, la caché KV mantiene en contexto la consulta del usuario y el plan de actuación, lo que guía al modelo durante la generación de la respuesta final.
- Durante el entrenamiento, el modelo se refuerza con pares de ejemplos *prompt*-planificación y planificación-respuesta.

La intuición: Razonamiento extendido

- Ejemplo de razonamiento extendido:



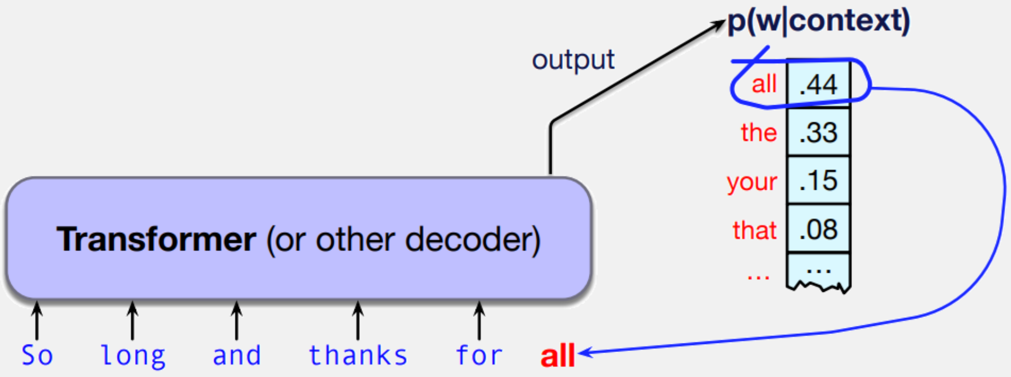
```
david@ZephyrusG14:~$ ollama run gemma4:e4b
>>> Resume brevemente el proceso de razonamiento extendido en un modelo LLM
Thinking...
Here's a plan to construct the answer:
1.  **Identify the core concept:** The user is asking for a brief summary of the "Extended Reasoning Process"
(Razonamiento Extendido) within an LLM.
2.  **Determine the key mechanism:** "Extended Reasoning" goes beyond simple next-token prediction. It involves
breaking down problems, planning, and simulating steps.
3.  **Structure the explanation (Briefly):**
    *   *Definition:* What is it? (Moving from statistical prediction to structured problem-solving).
    *   *Mechanism/How it works:* How does the LLM achieve this? (Step-by-step thinking, internal monologue, tool
use).
    *   *Key Techniques:* Name the common methods (CoT, ReAct, Planning).
    *   *Goal:* What is the outcome? (Increased accuracy, handling complexity).
4.  **Draft the response (in Spanish, maintaining a professional/technical tone):** (Self-Correction during
drafting: Ensure the explanation is *brief* but comprehensive.)

*(Execute drafting)*
...done thinking.

El **razonamiento extendido** (Extended Reasoning) en un Modelo de Lenguaje Grande (LLM) se refiere a la capacidad
de la IA para ir **más allá de la simple predicción de la siguiente palabra (token)** y simular un proceso de
pensamiento estructurado, desglosando problemas complejos en pasos lógicos manejables.

En esencia, no solo *sabe* la respuesta; *muestra cómo llegó* a ella.

Aquí se resume el proceso en tres etapas clave:

---
```

3

Generación de texto

Generación: Selección del siguiente *token*

- Los modelos LLM procesan el contexto disponible, generando una **distribución de probabilidades** de la adecuación de cada *token* del vocabulario del modelo como aquél a predecir.
- Debemos **determinar qué *token* seleccionamos** en base a la distribución obtenida tras procesar el contexto:

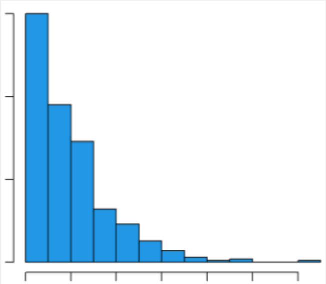

p(w|context)
output
all .44
the .33
your .15
that .08
… …
Transformer (or other decoder)
So
long
and
thanks
for
all

Generación: Selección del siguiente *token*

- Los modelos LLM procesan el contexto disponible, generando una distribución de probabilidades de la adecuación de cada *token* del vocabulario del modelo como aquél a predecir.
- Debemos determinar qué *token* seleccionamos en base a la distribución obtenida tras procesar el contexto:
  - **Decodificación avariciosa (*greedy*)**: Seleccionar sistemáticamente el *token* que tiene la probabilidad más alta.
    - Ventaja: El resultado es (generalmente) coherente.
    - Problema: Los resultados suenan muy genéricos, planos y repetitivos.
    - 100% determinista, un mismo *prompt* genera siempre la misma respuesta.
  - **Decodificación por muestreo (*sampling*)**: Realizamos una selección aleatoria en base a la probabilidad de cada término.
    - Ventaja: El texto parece más humano, es más diverso y creativo.

Generación: El problema del muestreo aleatorio

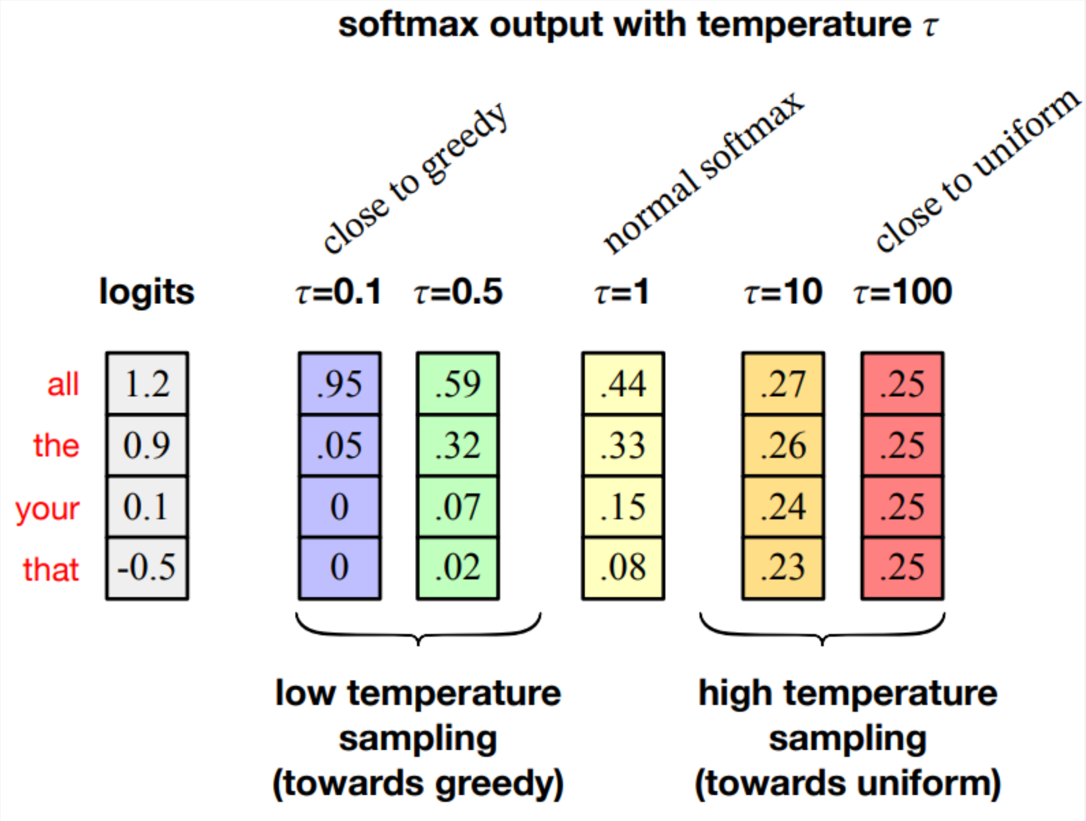
- El muestreo aleatorio puro presenta un **riesgo matemático grave** debido a la inmensidad del vocabulario de un modelo LLM (ej: 50k – 100k *tokens*).
- Generalmente, **sólo un puñado de *tokens* (ej: 2-5) son adecuados**, y son aquellos que acumulan la mayor parte de la probabilidad (ej. 70%).
- A pesar de ello, existen decenas de miles de palabras no tan adecuadas que, aunque cada uno tenga una probabilidad minúscula individual (ej. 0.001%), **la suma total de las probabilidades de todos ellos puede obtener un porcentaje representativo del total** (ej: 20-30%).
- Resultado: En el muestreo aleatorio puro, **existe un riesgo estadístico inaceptablemente alto de seleccionar un término no adecuado**, forzando al modelo a generar una palabra sin sentido que arruine el texto a generar. Este término **se añadirá al contexto, condicionando además la generación de los siguientes términos**.



Generación: El parámetro de temperatura

- Para evitar este problema sin perder creatividad, modificamos la distribución de probabilidades:
  - Los modelos generan **logits** para cada término del vocabulario, que se transforman en probabilidades a través de la función softmax.
  - Para adaptar la distribución de probabilidades, dividimos cada logit por un parámetro denominado **temperatura** (τ, $0 < \tau < \infty$) antes de aplicar softmax.
- Efecto del parámetro temperatura:
  - **Temperatura baja ($\tau < 1$) - “enfriar“, menor creatividad**: Al dividir el logit por un decimal, los logits aumentan su valor absoluto. La función softmax exagera las diferencias entre las probabilidades de los términos: las probabilidades altas se elevan mucho y las bajas caen todavía más. El modelo se vuelve más estable, determinista y "*greedy*".
  - **Temperatura alta ($\tau > 1$) - “calentar“, mayor creatividad**: Al dividir el logit por un número grande, los logits se acercan a cero. Como resultado, la diferencia entre las probabilidades de los términos se reduce, lo que aumenta la creatividad de las respuestas.

Generación: El parámetro de temperatura



softmax output with temperature τ

close to greedy
normal softmax
close to uniform

	logits	**τ=0.1**	**τ=0.5**	**τ=1**	**τ=10**	**τ=100**
all	1.2	.95	.59	.44	.27	.25
the	0.9	.05	.32	.33	.26	.25
your	0.1	0	.07	.15	.24	.25
that	-0.5	0	.02	.08	.23	.25

low temperature sampling (towards greedy)

high temperature sampling (towards uniform)

Generación: Filtrado de la distribución de probs.

- Ajustar la temperatura nos permite modificar fácilmente la creatividad del modelo sin necesidad de reentrenarlo, pero no soluciona necesariamente el problema original (generación de texto no coherente).
- Para eliminar este problema, **debemos filtrar los términos no adecuados** antes del muestreo:
  - **Top-k *sampling***: Se ordenan los términos por probabilidad y se retienen únicamente las k opciones más probables (ej. k=40). El resto de términos ven su probabilidad forzada a 0%.
  - **Top-p *sampling* (*nucleus sampling*)**: En lugar de un número fijo de palabras, se establece un umbral de masa de probabilidad p (ej. p=0.90). Se van sumando las probabilidades de los términos más probables hasta alcanzar el umbral. El modelo realiza una selección aleatoria exclusivamente entre los términos de ese "núcleo" lógico, ignorando los demás.
- **Ventaja de top-p *sampling***: Se adapta dinámicamente. El núcleo podrá estar compuesto de un par de términos o de un centenar en función de si está muy seguro o no del término a predecir.

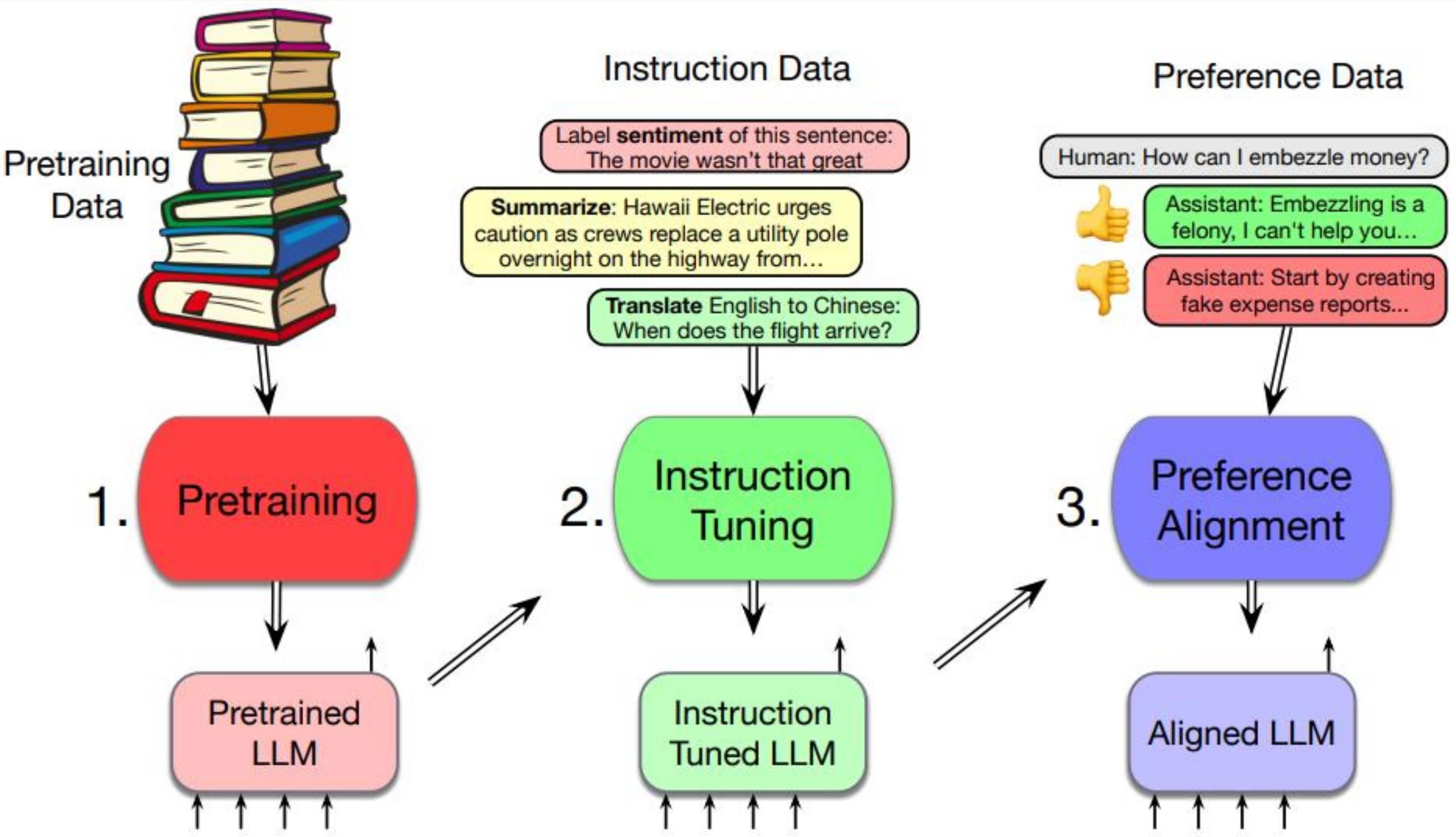
4

Entrenamiento

Entrenamiento: Fases de entrenamiento

- El entrenamiento de los modelos LLM consta de tres fases:
    1. **Preentrenamiento (*pre-training*)**: El objetivo de esta fase es que el modelo **aprenda las reglas del lenguaje**. Esto se consigue mediante entrenamiento auto-supervisado con terabytes de texto, buscando predecir iterativamente la siguiente palabra en el texto. El resultado es un modelo base.
    2. **Ajuste por instrucciones (*instruction tuning*)**: Busca adaptar la manera en la que el modelo genera el texto para **construir un sistema orientado a obedecer las órdenes del usuario**. Se entrena el modelo base con miles de ejemplos estructurados como "Usuario: [Prompt] -> Modelo: [Respuesta]", que emulen diferentes consultas de usuarios, con formatos de salida adaptados a las mismas.
    3. **Alineación de preferencias (*preference alignment*)**: Refinamiento final, donde se entrena al modelo mediante aprendizaje por refuerzo **utilizando las respuestas que gustan o no a los usuarios**.

Entrenamiento: Fases de entrenamiento


Pretraining Data
Instruction Data
Preference Data
1.
Pretraining
2.
Instruction Tuning
3.
Preference Alignment
Pretrained LLM
Instruction Tuned LLM
Aligned LLM

Label **sentiment** of this sentence: The movie wasn't that great

Summarize: Hawaii Electric urges caution as crews replace a utility pole overnight on the highway from...

Translate English to Chinese: When does the flight arrive?

Human: How can I embezzle money?

Assistant: Embezzling is a felony, I can't help you...

Assistant: Start by creating fake expense reports...

Entrenamiento: Preentrenamiento

- Durante el preentrenamiento, el modelo se entrena con TB de datos textuales:
  - Common crawl / C4: 156B *tokens* (volcados de WEBs).
  - The pile: 825 GB de datos estructurados (libros, internet, ArXiv, GitHub).
  - Wikipedia, libros, subtítulos de películas/series, artículos, código…

- Desafíos:
  - Filtrado de calidad, deduplicación, filtrado de ruido.
  - Eliminación de textos con información peligrosa/ilegal y comentarios tóxicos.
  - Sesgo (sobrerrepresentaciones).
  - Copyright, consentimiento (*scraping*) y PII.

Entrenamiento: Preentrenamiento

- Procedimiento de preentrenamiento:
  - Se ***tokenizan*** los documentos utilizando *tokenización* de subpalabras (BPE, SentencePiece, etc.).
  - Se construyen ***mini-batches* de B secuencias de T *tokens***, siendo T el tamaño de contexto del modelo.
  - Para **cada posición dentro de cada secuencia del *mini-batch***, se obtiene la predicción de la **distribución de probabilidades de siguiente *token***. Se aplica una **máscara sobre *tokens* futuros para asegurar que cada posición solo atiende a *tokens anteriores***.
  - Calculamos la **entropía cruzada negativa** entre la distribución real (*one-hot*, 1 para el *token* real 0 para el resto) y la distribución predicha, para cada elemento del *mini-batch* (cada predicción de siguiente *token* en cada posición de cada secuencia):
    - $\mathcal{L}_{b,t} = -\sum_x p_{real,b,t}(x) \log\big(p_{modelo}(x|\, x_{b,\le t})\big) = -\log\big(p_{modelo}\big(x_{b,t+1} \,\big|\, x_{b,\le t}\big)\big)$
  - Obtenemos el error medio del *mini-batch* como:
    - $\mathcal{L} = \frac{1}{N}\sum_{b=1}^{B}\sum_{t=1}^{T-1}\mathcal{L}_{b,t} = -\frac{1}{N}\sum_{b=1}^{B}\sum_{t=1}^{T-1}\log(p_{modelo}(x_{t+1} \mid x_{\le t}))\,,\quad N = B\cdot(T-1)$

Entrenamiento: Preentrenamiento

- Procedimiento de preentrenamiento:
  - Una vez calculada la pérdida media del *mini-batch*, se calcula el **gradiente de $\mathcal{L}$ respecto a cada parámetro** del modelo mediante backpropagation:
    - Calculamos el gradiente de cada una de las distribuciones de probabilidad obtenidas (para cada *token* en cada secuencia) **respecto a los logits de cada posición**:
      - $\frac{\partial \mathcal{L}_{b,t}}{\partial z_{b,t}[k]} = p_{modelo}\left(x_{b,t+1} = k \mid x_{b,\leq t}\right) - 1\{k = x_{b,t+1}\})$
      - (es decir, gradiente = predicción − realidad).
    - Utilizamos ***backpropagation*** para calcular la contribución de cada parámetro del modelo al error. Promediamos los gradientes individuales para obtener el gradiente total del *mini-batch*:
      - $\mathrm{g} = \nabla_\theta \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial \theta_1}, \frac{\partial \mathcal{L}}{\partial \theta_2}, \ldots\right) = \frac{1}{N} \sum_{b,t} \nabla_\theta \mathcal{L}_{b,t}$
  - **Ajustamos los pesos** del modelo en base a este gradiente para reducir el error:
    - Ejemplo: θ_{t+1}=θ_t - $\eta \cdot \mathrm{g}$ (en realidad, se utilizan mecanismos más complejos como AdamW).

Entrenamiento: Ajuste por instrucciones

- El procedimiento de **ajuste por instrucciones** es similar al de preentrenamiento, pero se introducen los siguientes cambios:
  - Entrenamos el modelo a través de **ejemplos representativos de *prompt*-respuesta**:
    - Ej: Usuario: "¿Cuál es la capital de Francia?" -> Modelo: "París.“
    - Que pueden dar lugar a la siguiente lista de *tokens*: <|usuario|> "¿Cuál" "es" "la" "capital" "de" "Francia?" <|asistente|> "París." <|fin|>
  - **No nos interesa que el modelo aprenda a predecir el *prompt*** del usuario, sólo que mejore su respuesta en base al *prompt* del usuario. Aplicamos el mismo procedimiento que en preentrenamiento, pero **establecemos a 0 el error de todos los *tokens* que no correspondan a la respuesta esperada del modelo**.
  - Como resultado de este proceso, el modelo **aprende formatos de respuesta adaptados a diferentes tareas** (resumen, clasificación…).

Entrenamiento: Alineación de preferencias

- La **alineación de preferencias** busca adaptar el proceso de generación de texto a las necesidades o preferencias de los usuarios.
- Utilizamos un conjunto de datos basado en el *feedback* de los usuarios para mejorar el modelo preentrenado:
  1. Construimos un **modelo de recompensa** basado en una red neuronal, que aprenda a **calificar la satisfacción esperada del usuario sobre una respuesta**, en base al *feedback* explícito obtenido. El modelo tomará pares *prompt*-respuesta, y se entrenará con ternas *prompt*-respuesta positiva-respuesta negativa, ajustando sus pesos para generar una valoración superior para la respuesta valorada positivamente frente a la valorada negativamente.
  2. Utilizamos un **algoritmo de aprendizaje por refuerzo**, que genere texto a través del LLM usando *prompts* de la base de datos de *feedback*. Este modelo evaluará los resultados a través del modelo de recompensa y **ajustará directamente los pesos del modelo** para tratar de obtener respuesta con una mayor valoración.

Entrenamiento: Fine-tuning

- El término **fine-tuning** se refiere a cualquier proceso en el que se toma un modelo LLM preentrenado para seguir ajustando sus pesos a través de un conjunto de datos mucho más pequeño y especializado.
- Aplicaciones:
  - **Adaptación a dominio**: Aprender patrones lingüísticos de un dominio específico, inyectando vocabulario y relaciones que reflejan conocimientos específicos.
  - **Adaptación a tarea**: Aprender patrones vinculados con formatos de texto (tanto de los *prompt* como de la respuesta), que ayuden a completar adecuadamente una tarea.
- Debido al tamaño de las matrices de pesos de los modelos LLM, reentrenar un modelo es un proceso **extremadamente costoso** (memoria y cómputo).

Entrenamiento: Fine-tuning (LoRA)

- PEFT (*parameter-efficient fine-tuning*) es un conjunto de técnicas que facilitan el proceso de *fine-tuning*. La técnica más popular dentro de PEFT es **LoRA** (*low-rank adaptation*).
- Cuando aplicamos LoRA, **congelamos las matrices de pesos originales** del modelo LLM y **construimos dos matrices A,B complementarias de tamaño reducido**, cuyo producto tiene las mismas dimensiones que la matriz de pesos original.
- De esta manera:
  - $W_{LoRA} = W_{original} + (A \cdot B)$
- El proceso de entrenamiento a través de LoRA es el mismo que el original. Dado que las matrices LoRA A,B tienen un tamaño muy reducido, el proceso de entrenamiento es más rápido y se reduce significativamente la memoria VRAM necesaria.

5

Evaluación

Evaluación: Verosimilitud y perplejidad

- ¿Cómo evaluamos el rendimiento de un modelo LLM?
  - **Verosimilitud (*likelihood*)**: Sumamos los logaritmos de las probabilidades de cada uno de los términos seleccionados por el modelo:
    - $log - likelihood(x_{1:n}) = \sum_{i=1}^{n} \log(p(x_i|x_{<i}))$
    - Problema: La verosimilitud depende de la longitud del texto: secuencias más largas tienden a tener peor puntuación (más negativa), aunque el modelo sea igual de bueno.
  - **Perplejidad (*perplexity*, PPL)**: Mide el grado de sorpresa del modelo durante la predicción de *tokens*:
    - $PPL(x_{1:n}) = e^{\left(-\frac{1}{n}\sum_{i=1}^{n} \log(p(x_i|x_{<i}))\right)}$
    - Este valor se puede interpretar como el número efectivo de opciones equiprobables que producirían el mismo nivel de incertidumbre.

Evaluación: Perplejidad

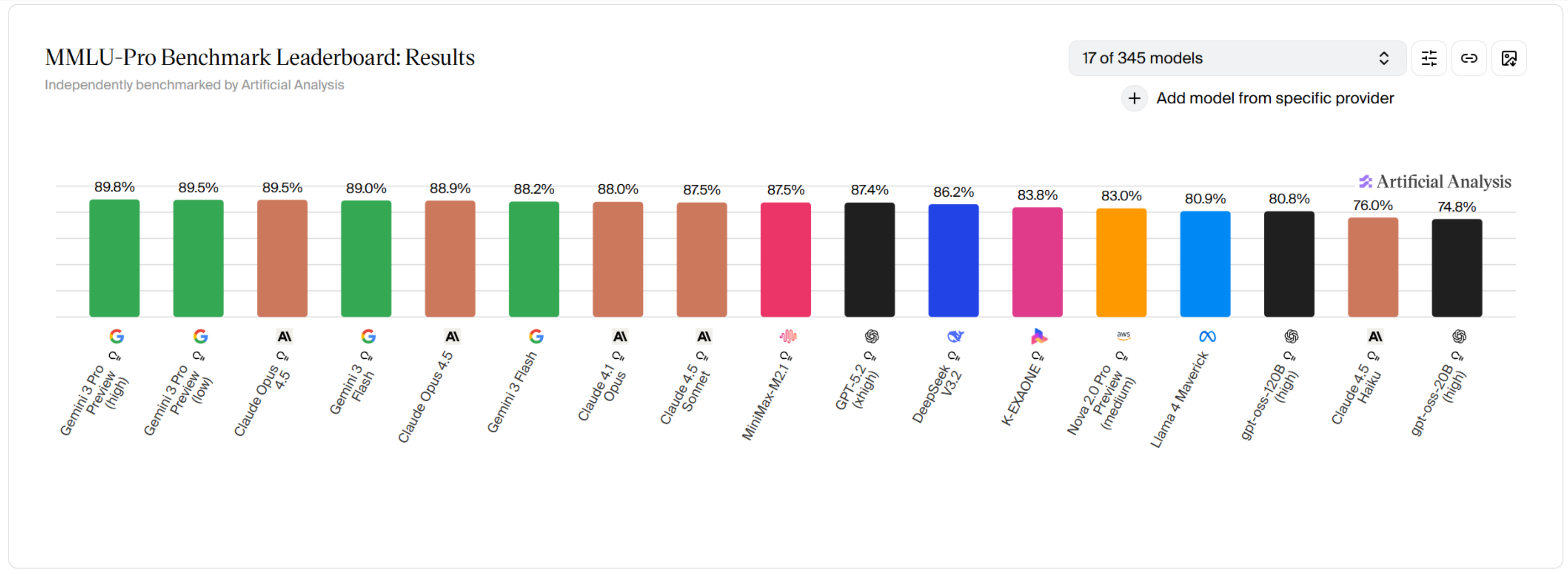
- Ejemplo:
  - Dadas las siguientes probabilidades de los términos 1:3 elegidos por el modelo:
    - [0.8, 0.4, 0.25]
  - La suma de los logaritmos (neperianos) negativos es:
    - -ln(0.8) –ln(0.4) –ln(0.25) = 0.2231 + 0.9163 + 1.3863 = 2.5257
  - La entropía cruzada negativa promedio es, por tanto:
    - $\frac{2.52572}{3} = 0.8419$
  - La perplejidad es, por tanto:
    - $PPL = e^{0.8419} = 2.3207$
  - Este valor indica que el modelo tiene una incertidumbre equivalente a elegir entre ~2.3 opciones equiprobables en cada predicción.

Evaluación: Otras métricas de evaluación

- Otras métricas habituales utilizadas para evaluar un modelo LLM son:
  - **Métricas de eficiencia y despliegue**:
    - Tiempo hasta generar el primer *token* (TTFT).
    - Velocidad de inferencia (*tokens*/s).
    - Tamaño en VRAM.
    - Coste energético de inferencia.
  - **Métricas de evaluación extrínseca (*benchmarks*)**:
    - MMLU: Batería masiva de preguntas de conocimiento general multidisciplinar.
    - HumanEval: Test para medir la capacidad del modelo para escribir código de programación funcional.
    - GSM8K: Benchmark estándar para evaluar la capacidad de razonamiento lógico y matemático.
    - Evaluación humana (Elo/arena): Sistemas de ranking basados en encuestas a ciegas donde humanos eligen qué modelo da la mejor respuesta.

Evaluación: Otras métricas de evaluación

- Clasificación MMLU:



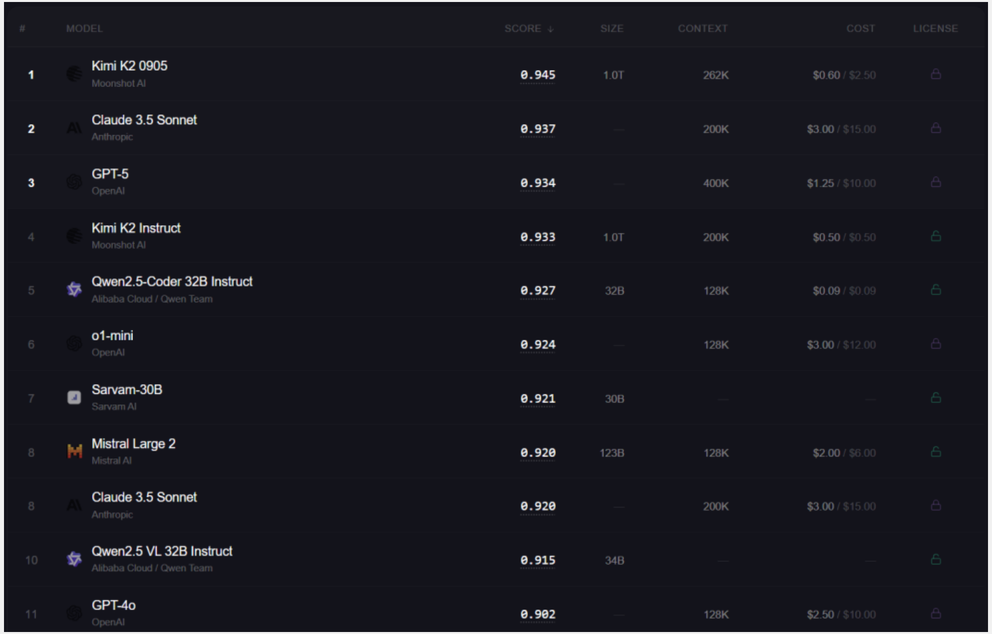
MMLU-Pro Benchmark Leaderboard: Results

Independently benchmarked by Artificial Analysis

- Fuente: https://artificialanalysis.ai/evaluations/mmlu-pro

Evaluación: Otras métricas de evaluación

- Clasificación HumanEval:

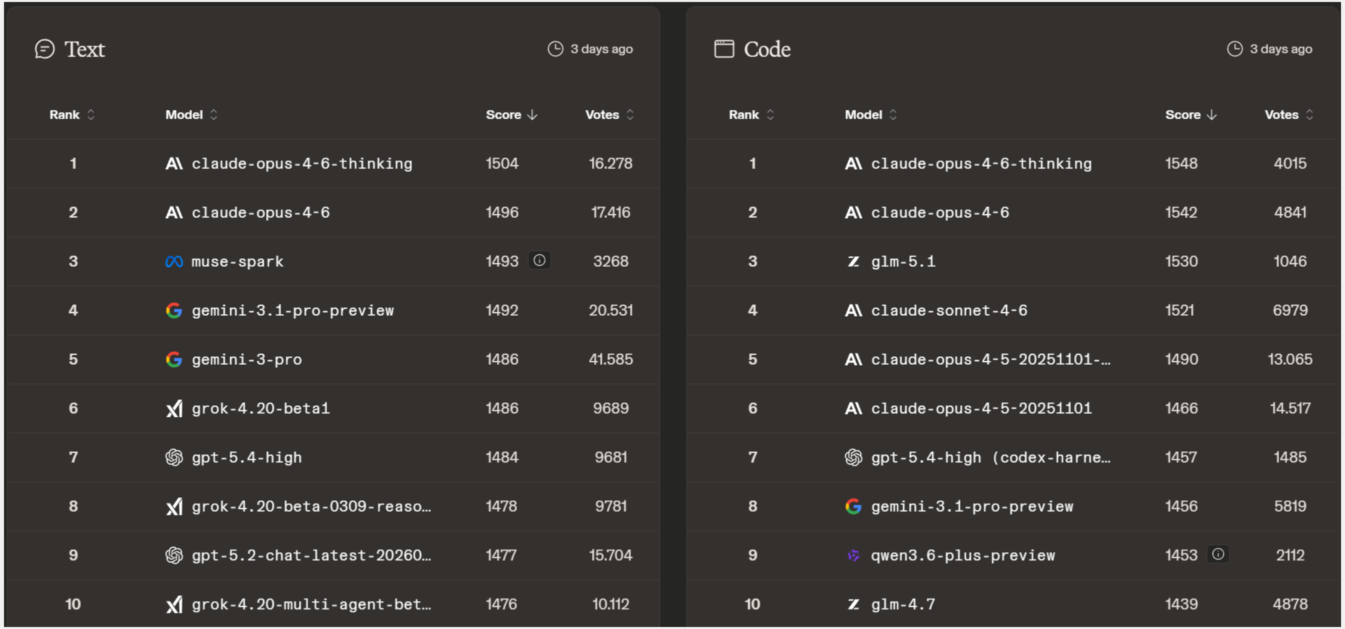


#	MODEL	SCORE ↓	SIZE	CONTEXT	COST	LICENSE
1	Kimi K2 0905 Moonshot AI	0.945	1.0T	262K	$0.60 / $2.50	🔒
2	Claude 3.5 Sonnet Anthropic	0.937	—	200K	$3.00 / $15.00	🔒
3	GPT-5 OpenAI	0.934	—	400K	$1.25 / $10.00	🔒
4	Kimi K2 Instruct Moonshot AI	0.933	1.0T	200K	$0.50 / $0.50	🔓
5	Qwen2.5-Coder 32B Instruct Alibaba Cloud / Qwen Team	0.927	32B	128K	$0.09 / $0.09	🔓
6	o1-mini OpenAI	0.924	—	128K	$3.00 / $12.00	🔒
7	Sarvam-30B Sarvam AI	0.921	30B	—	—	🔓
8	Mistral Large 2 Mistral AI	0.920	123B	128K	$2.00 / $6.00	🔓
8	Claude 3.5 Sonnet Anthropic	0.920	—	200K	$3.00 / $15.00	🔒
10	Qwen2.5 VL 32B Instruct Alibaba Cloud / Qwen Team	0.915	34B	—	—	🔓
11	GPT-4o OpenAI	0.902	—	128K	$2.50 / $10.00	🔒

- Fuente: https://llm-stats.com/benchmarks/humaneval

Evaluación: Otras métricas de evaluación

- Clasificación Elo:



Text

3 days ago

Rank	Model	Score	Votes
1	claude-opus-4-6-thinking	1504	16.278
2	claude-opus-4-6	1496	17.416
3	muse-spark	1493	3268
4	gemini-3.1-pro-preview	1492	20.531
5	gemini-3-pro	1486	41.585
6	grok-4.20-beta1	1486	9689
7	gpt-5.4-high	1484	9681
8	grok-4.20-beta-0309-reaso...	1478	9781
9	gpt-5.2-chat-latest-20260...	1477	15.704
10	grok-4.20-multi-agent-bet...	1476	10.112

Code

3 days ago

Rank	Model	Score	Votes
1	claude-opus-4-6-thinking	1548	4015
2	claude-opus-4-6	1542	4841
3	glm-5.1	1530	1046
4	claude-sonnet-4-6	1521	6979
5	claude-opus-4-5-20251101-...	1490	13.065
6	claude-opus-4-5-20251101	1466	14.517
7	gpt-5.4-high (codex-harne...	1457	1485
8	gemini-3.1-pro-preview	1456	5819
9	qwen3.6-plus-preview	1453	2112
10	glm-4.7	1439	4878

- Fuente: https://arena.ai/leaderboard

¿Preguntas?